08
Agile Software Development

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

1. Overview of processes
2. Agile software development
3. The case of Scrum

Summary

Processes in software engineering

- they describe how teams are organised to develop a system
- which tasks are appointed to people
- the workflow of development activities

Outline

- waterfall as the (unused) core framework
- variations and other process types
- agile methodologies
- Scrum

Waterfall

Idea

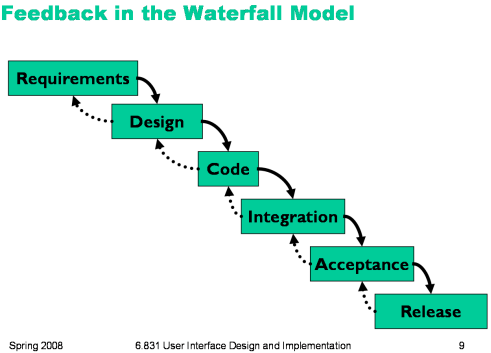
- consider the main stages of classical software engineering
- start one only when the preceding is completed
- possibly, have each phase developed by a different team
- strong focus on the artifacts produced in each stage
- it's known to be a predictive method: it requires to foresee the future

Macro-Stages

- Requirements engineering ⇒ Specification: which behaviour fulfill reqs?
- Architectural design ⇒ System architecture: which software macro structure?
- Detailed design ⇒ Detailed design document: which software micro structure?
- Implementation ⇒ First working code: which runnable code?
- Testing and maintenance ⇒ Correct & tested code: which reqs are met?
- Deployment ⇒ Running system: is the system successfully online?

Waterfall variants: waterfall with feedbacks

1

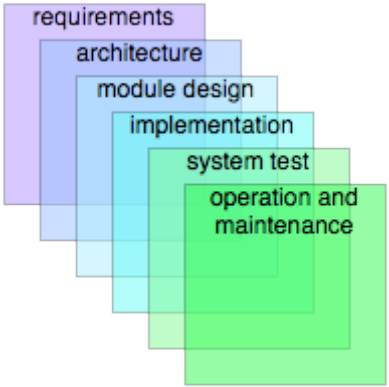


Key elements

- the process is structured to consider "going back" as "the norm"
- during each phase, the result of previous one can be revised

Waterfall variants: waterfall with overlapping phases

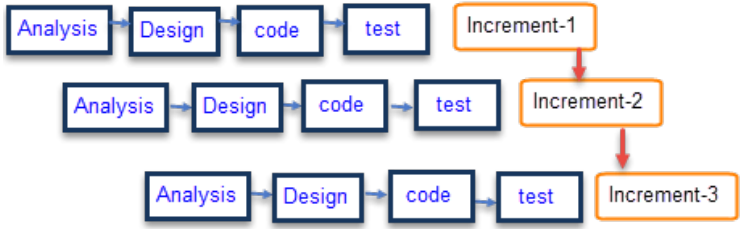
2



Key elements

- the process is structured to have smooth transition phases
- the different teams keep interacting throughout

Waterfall variants: incremental waterfall



Incremental Model

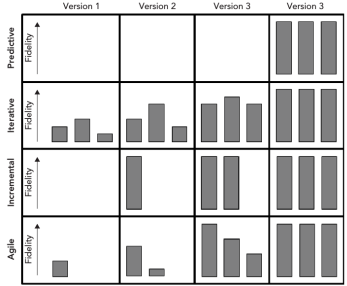
Key elements

- the process is structured in isolated "increments"
- implement pieces/releases one by one, at different temporal stages

Comparison of process model types

Differences

- Predictive: provide all the features at the same time and full accuracy
- Iterative: initially provides all features with partial accuracy, later versions increase accuracy
- Incremental: initially provides some features with full accuracy, later add features
- Agile: initially provides some features with partial accuracy, later add features and accuracy



Iterative/incremental models together: agility

Motivations and consequences

- build early version of the systems to get feedbacks
- feedbacks coming both from stakeholders and internally

Building early prototypes

- horizontal vs vertical prototypes
- pros: improving requirements, common vision, better design
- cons: narrowing vision, impatience, pressure, expectations, code-orientation

Outline

1 Overview of processes

2 Agile software development

3 The case of Scrum

Agile software development

The Agile manifesto initiative

- `http://agilemanifesto.org/`, 2001
- signed by many practitioners, the Agile Alliance
- deeply influenced (at least, in part) all software development
- started from criticising waterfall model
- defined by 12 principles
- key idea: do not refuse change, rather embrace it

Agile: a set of guidelines, not a true process

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter time-scale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of ~~conveying~~ trasmettere information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant ~~pace~~ ritmo indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity – the art of maximizing the amount of work not done – is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

Example agile methodologies

Lean

- a translation of lean manufacturing of lean IT principles to software development
- it seeks to highest efficiency
- principles: eliminate waste, amplify learning, decide as late as possible, deliver as fast as possible, empower the team, build quality in, see the whole

XP

- a set of core software engineering practices essential to every agile project
- (release planning, iteration planning, acceptance test, stand up meeting, pair negotiation, TDD, pair programming, code quality)

Scrum

- a Project management wrapper for agile projects
- ~~dictates~~ definisce roles (e.g., product owner), artefacts (e.g., product backlog), and meetings (e.g., Sprint retrospective)

What actually happens in industries

- most industries claim they use "agile methods" at least in part
- typically they just inherit some principle but fear to embrace them all
- Scrum is considered as the ultimate approach... but needs trust, experience and tailoring: some industries apparently use a good 80-90% of it

Outline

1 Overview of processes

2 Agile software development

3 The case of Scrum

Scrum

What is it

- an iterative and incremental agile software development framework for managing product development
- it defines "a flexible, holistic product development strategy where a development team works as a unit to reach a common goal"
- enables teams to self-organize by encouraging physical co-location, daily face-to-face, or close online collaboration of all team members

Origin

- paper: "The New Product Development Game" by Hirotaka Takeuchi and Ikujiro Nonaka (1986)
- Jeff Sutherland and Ken Schwaber separately start using the approach in their companies, and then produce a new paper coining the term "Scrum" (1993)
- Scrum stands for Italian "Mischia" in rugby

Our treatment

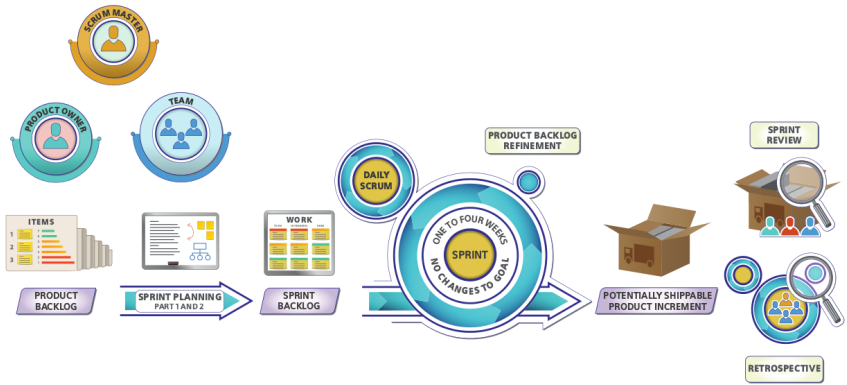
- will describe its main characteristics (see "The Scrum Primer 2.0")
- will later propose a Scrum-inspired process for the exam project

Scrum focus

Key answered questions

- how to break a long (months) development process in pieces/milestones
- how to split a large ($\geq$5 developers) developer process to the team members
- how to regulate relationships between team, product manager, and stakeholders
- how to manage selection of priorities throughout
- how to track evolution and state of development

Overview


SCRUM MASTER
PRODUCT OWNER
TEAM
ITEMS
PRODUCT BACKLOG
SPRINT PLANNING
PART 1 AND 2
WORK
SPRINT BACKLOG
DAILY SCRUM
PRODUCT BACKLOG REFINEMENT
ONE TO FOUR WEEKS
SPRINT
NO CHANGES TO GOAL
POTENTIALLY SHIPPABLE PRODUCT INCREMENT
SPRINT REVIEW
RETROSPECTIVE

Roles – product owner

Goal: a "one person" responsible for maximizing return on investment

- interacts with customers
- identifies product features
- creates a priority list
- maintains the list

Giving a semantic to "value", impacting priority...

- commercial value
- customer satisfaction
- strategic objectives
- attacking risks

A sort of Product Manager, with a notable difference..

- a PO definitely understands product development

$\Rightarrow$ continuously interacting with the team

$\Rightarrow$ reviewing the result of Sprints

$\Rightarrow$ is not accountable for fine-grained decisions

Roles – (development) team

Goals

- to build the product that the PO indicates
- to provide ideas to the PO
- to estimate time, and decide which items implement in a Sprint

Characteristics of the team

- "cross-functional" and "multi-learning"
- self-organising, autonomous and accountable
- 100% dedicated to a single project during the Sprint

Shape

- the team is made by 5-9 people, with no internal structure
- typically teams are stable during one or more projects
- team members altogether have various skills
  - analysis, development, testing, interface design, database design, architecture, documentation
- big projects are organised in flat "feature teams"

Roles – scrum master

Goals

- helps the group (PO, team, organisation) learn and apply Scrum
- he/she is not a leader, manager, or representative: but rather a coach
- typically not needed if the team is very experienced
- typically useful in moving towards an agile context
- it may help smooth coordination between PO and Team

Who serves as SM?

- not the PO, which would otherwise be a micro-managing one
- a person very skilled in Scrum
- possibly, an external ~~counsellor~~ consulente (there are courses and certifications)

Product backlog

..an artifact of Scrum, a sort of product development roadmap/table

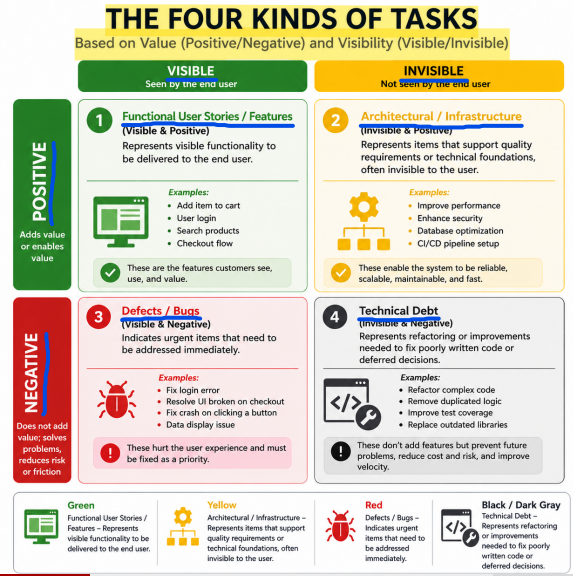
- a prioritised list of customer-centric features
- it exists and evolves over the project lifetime
- it is prioritised, emergent and **deep** (more fine-grained on top)
- each item comes with the following properties (columns in a table)
  - priority position, description, possibly URL to details, estimation size
  - estimation should be a relative number, say *points*.. team actual velocity (points/Sprint) emerge

Items/features in the product backlog board

- primarily, new customer features ("enable GUI-oriented interface")
- major engineering improvements ("Java2Scala rewrite of MVC-model")
- research work ("investigate approaches to develop game AI")
- improvement goals ("speed up tests")
- correcting major known defects ("make AI correctly decide the first move")
- bootstrap activities ("setting up the build environment", "design architecture")

$\Rightarrow$ agility requires that most items focus on delivering value to customers

Kinds of tasks



Product backlog example

				New Estimates at Sprint ...					
Priority	**Item**	**Details (wiki URL)**	**Initial Size Estimate**	**1**	**2**	**3**	**4**	**5**	**6**
1	As a buyer, I want to place a book in a shopping cart (see UI sketches on wiki page)	...	5						
2	As a buyer, I want to remove a book in a shopping cart	...	2						
3	Improve transaction processing performance (see target performance metrics on wiki)	...	13						
4	Investigate solutions for speeding up credit card validation (see target performance metrics on wiki)	...	20						
5	Upgrade all servers to Apache 2.2.3	...	13						
6	Diagnose and fix the order processing script errors (bugzilla ID 14823)	...	3						
7	As a shopper, I want to create and save a wish list	...	40						
8	As a shopper, I want to to add or delete items on my wish list	...	20						

Definition of done

When is an item/feature closed?

- items not finished need to be moved to a ~~forthcoming~~ following Sprint
- an item is closed when it is definitely "done"
- what is the definition of "done" must be agreed at the beginning
- it impacts the notion of "potentially shippable increment"

Definition of "done".. possible facets

- software that comes with thorough and non-failing tests
- software that does not break existing code
- software that is documented in certain ways
- software that comes with tutorials
- software that meets certain quality criteria
- software that is acceptance-tested

Activities of a Sprint

Whole Sprint (1-4 weeks) – the smaller, the more you are agile

- Sprint Planning (part 1 and 2): 1 hour x part x week of Sprint
- Daily Scrum: max 15 minutes
- Daily Scrum
- ...
- Daily Scrum
- Product Backlog Refinement: max 10% of Sprint time, sometimes heavier
- Sprint Review: 1 hour x week of Sprint
- Sprint Retrospective: 45 mins x week of Sprint

Sprint Planning – part one

Activity

- Summary — A meeting to prepare the Sprint (p.1: what, p.2: how)
- Participants — All the Scrum Team (in part 2 the PO is optional)

Part one: what to do

- reviewing of high-priority items (already analysed)
- discussion of goals and contexts of such items
- focussing on what the PO wants and why this is needed
- result: Sprint Goal
  - a summary statement of the Sprint objective
  - leaving flexibility to quantity and actual selection of items

Sprint planning – part two

Part two: how to implement

- forecasting the amount of items that can be completed
  - possibly considering the velocity so far
  - typically assuming a team member only works 4-6 hours per day
- starting from top of the product backlog
- possibly picking up items from further down the list
- crucially: decisions are taken by the team, not the PO
- the team commits to a "forecast", communicated to the PO

Sprint planning – Sprint backlog

Sprint Backlog

- a refinement of the portion of the Product Backlog for the given Sprint
- each item broken in several Sprint tasks, each with estimates
- ~~people task assignment~~ assegnazione delle persone ai compiti
  - not only based on ~~volunteering~~ disponibilità or skills
  - typically, only one task per person ~~following skill~~ in base alle competenze
  - assignment can also in part be dynamic
- often rendered in a board with post-it-notes ("to-do", "WIP", "done")
- changes to the Sprint target necessarily go into next Sprint
- dramatic changes need to stop current Sprint

Sprint planning – Sprint backlog

				New Estimates of Effort Remaining at end of Day...					
Product Backlog Item	**Sprint Task**	**Volunteer**	**Initial Estimate of Effort**	**1**	**2**	**3**	**4**	**5**	**6**
As a buyer, I want to place a book in a shopping cart	modify database		5						
	create webpage (UI)		8						
	create webpage (Javascript logic)		13						
	write automated acceptance tests		13						
	update buyer help webpage		3						
	. . .								
Improve transaction processing performance	merge DCP code and complete layer-level tests		5						
	complete machine order for pRank		8						
	change DCP and reader to use pRank http API		13						

Daily Scrum

Activity

- Summary — Update and coordination between the Team members
- Participants — Team required, PO optional, SM typically present
- When — Each day, for 15 minutes, at appointed time: team standing

Points of discussion

- synchronise work and report obstacles
- each member reports: (i) what is done, (ii) what will be done, (iii) what obstacles
- there's little or no in-depth technical discussion (it should follow in parallel follow-up meetings)

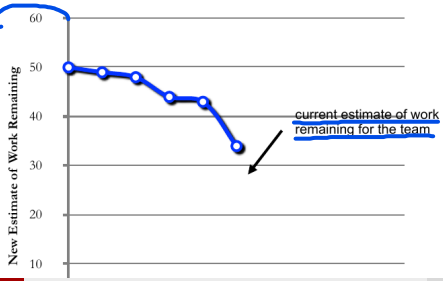
Tracking progress during the Sprint

- every day, updating the Sprint Backlog (with estimates and work done)
- plot the "Sprint burndown chart"

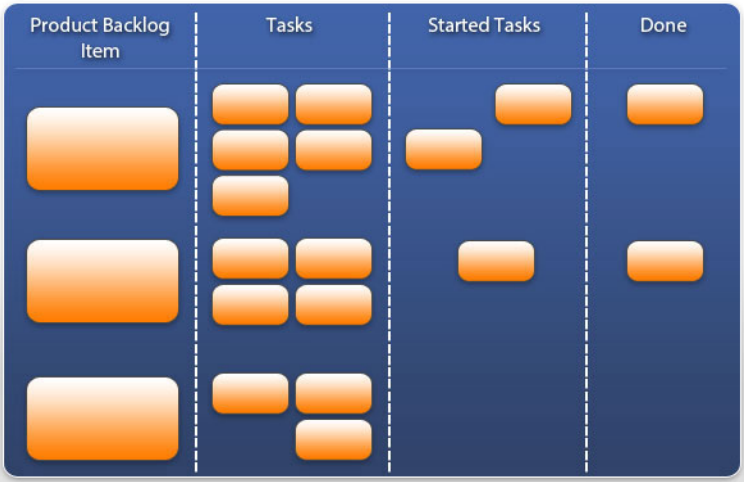
Tracking progress during a Sprint

				New Estimates of Effort Remaining at end of Day...					
Product Backlog Item	**Sprint Task**	**Volunteer**	**Initial Estimate of Effort**	**1**	**2**	**3**	**4**	**5**	**6**
As a buyer, I want to place a book in a shopping cart	modify database	Sanjay	5	4	3	0	0	0	
	create webpage (UI)	Jing	3	3	3	2	0	0	
	create webpage (Javascript logic)	Tracy & Sam	2	2	2	2	1	0	
	write automated acceptance tests	Sarah	5	5	5	5	5	0	
	update buyer help webpage	Sanjay & Jing	3	3	3	3	3	0	
	. . .								
Improve transaction processing performance	merge DCP code and complete layer-level tests		5	5	5	5	5	5	
	complete machine order for pRank		3	8	8	8	8	8	
	change DCP and reader to use pRank http API		5	5	5	5	5	5	

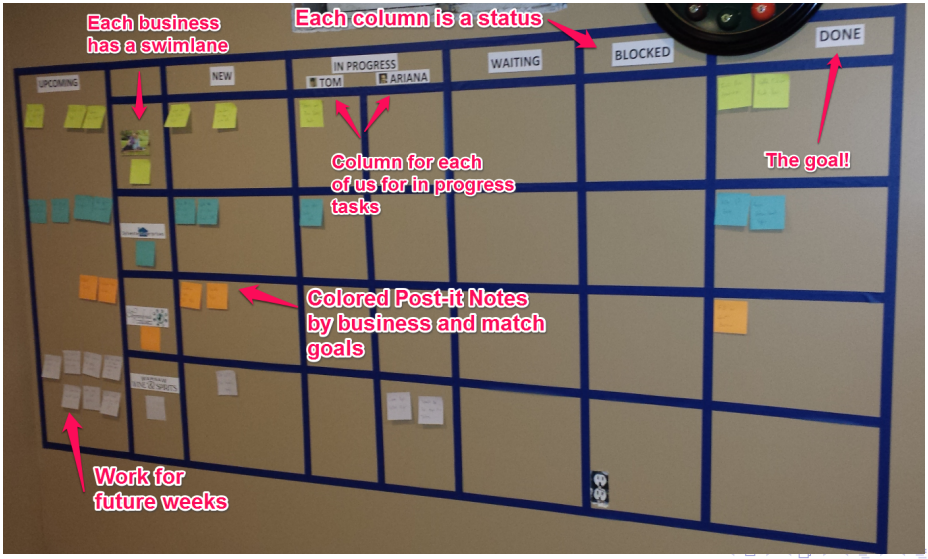
Un Sprint Burndown Chart è un grafico visivo per mostrare la quantità di lavoro rimanente in uno Sprint rispetto al tempo a disposizione:
- Asse X rappresenta il tempo (i giorni dello Sprint, dal giorno 0 fino all'ultimo giorno).
- Asse Y rappresenta il lavoro rimanente, misurato in story points, ore o numero di task.
- Linea ideale (ideal progress): Una linea retta che mostra l'andamento teorico costante del lavoro che dovrebbe essere svolto giorno per giorno fino a raggiungere lo zero.
- Linea reale (actual remaining work): La linea che traccia il progresso effettivo del team in base alle attività realmente completate.



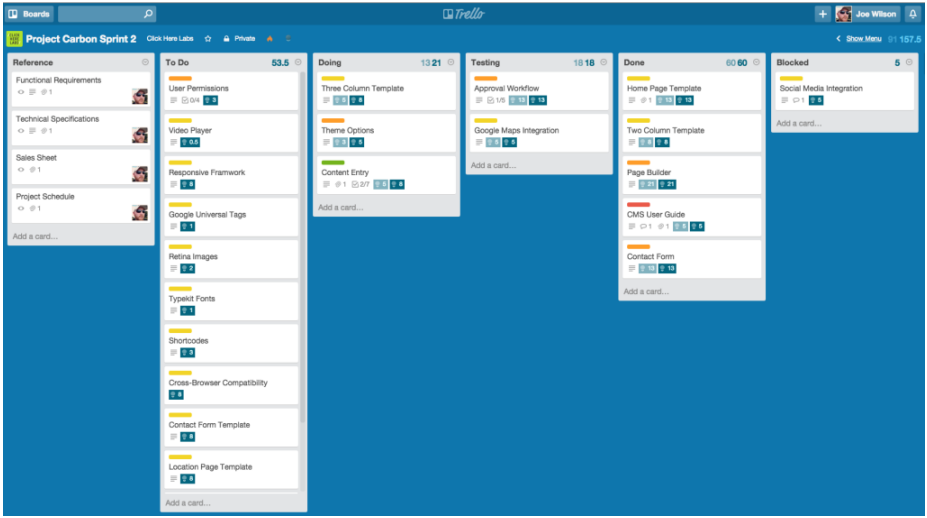
Sprint task board: content



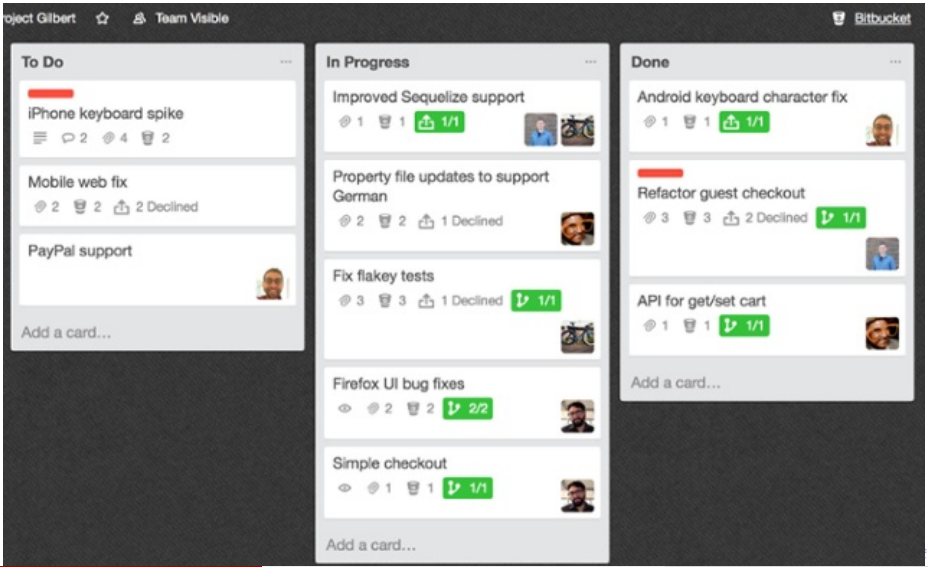
Sprint task board: on blackboard



Sprint task board: using Trello collaboration tool



Sprint task board: Trello integration with BitBucket



Product backlog refinement

Activity

- Summary — Split big items, analyse, re-estimate, re-prioritise for future Springs
- Participants — Team, PO, SM only in early Sprints

Points of discussion

- expanding and deepening for next 1/2 Sprints
- doing it properly allows Sprint Planning to be quick and effective

Sprint review

Activity

- Summary — Inspection and adaptation related to the product increment of functionality
- Participants — Whole Scrum Team, possibly also stakeholders and users

Points of discussion

- often mislabelled “demo”, which is not
- it is key for the PO to understand how the product is evolving
- it is key for the Team to learn about PO and market
- typically includes PO+Team conversation + using “live software”
- no formal presentation with slides or presentation demo

Activity

- Summary — Inspection and adaptation related to the process or environment
- Participants — Team, SM, PO (optional)

Points of discussion

- discussing what's working and not working
- discussing changes to try
- there's a whole catalogue of possible techniques (Agile retrospectives)
- should focus both on positive and negative strengths

Next Sprint Planning comes immediately after..

On the role of testing and continuous integration/delivery

Whole life-cycle

- Sprints come one after the other
- team effectiveness typically increases from Sprint to Sprint
- it is the PO that decides when and what to actually release
- proper technologies must be used for quick and effective delivery
  - if you do not have them, you need a "Release Sprint"

Continuous integration/delivery

Frameworks, techniques and tools to:

- quickly add new software to an existing codebase
- do (automatic) regression tests before releasing
- releasing new software automatically

A possible process for the exam project

Elements

- One student, for example the person who had the project idea, will act as a sort of client or domain expert, as well as a developer. He/she will try to ensure the usability and quality of the result.
- One student, for example someone who believes he/she have coordination skills, will act as Product Owner, as well as a developer.
- In an initial meeting, the team should prepare a Product Backlog and define a first organizational sprint. This sprint should cover preparation of the build, identification of basic requirements and architecture, and identification of the basic elements of the development process, such as sprint duration, Definition of Done, and the types and frequency of meetings.
- Sprints should be short, involving about 15 hours of work over one week, so that around four sprints can be completed in total.
- In each sprint, the team should try to obtain tangible results that already provide value for the stakeholders, namely the teacher or the student/client.
- The team should also maintain a Sprint Backlog, hold frequent meetings, and organize meetings at the beginning and end of each sprint. At the end of each sprint, a very brief report of the result should be produced and also kept under version control.